Config Inversion Library for Tracers

Goal Description

This RFC proposes the establishment of a **central configuration inversion library** that all tracers must reference for configuration definitions. The objective is to standardize tracer configuration behavior across a variety of programming languages (Go, PHP, Java, Python, .NET, Node.js, Ruby, etc.) and to enforce consistency via automated CI checks.

This makes sure that it's known what tracer supports what configuration without the need for any team to gather that data. This way a sales and support person is able to immediately answer what features are available in what programming language.

On top of that, it's possible to auto generate the documentation for users from the central configuration.

It also addresses the issue of supporting different implementations of a single configuration and alias handling.

Later on, this may be improved further to help out the fleet automation by adding information about the system requirements and many things more.

Detect which configurations are slightly different implementations and yet pretty much the same and should be unified. Document these.

Goal Summary

The central configuration resolves the following issues

1. We enforce consistent documentation of all configurations across all platforms (Node.js, Go, Dotnet, …)
2. The developer experience is improved or at least kept identical to the one today
3. Configuration handling across platforms is simplified
4. Automated alias handling
5. Later: Uniform configuration behavior across platforms

Problem Description

Each tracer currently implements configuration settings independently, potentially creating behavioral discrepancies. By defining a central configuration library, we eliminate these inconsistencies and provide:

- A **single source of truth** for all tracer configuration parameters.
- A **strict validation process** (via GitHub Actions) to ensure every tracer uses only known configurations.
- A **clear process** for adding or changing configurations and tracking version changes.
- A **source** that may be used to auto generate the internal and external documentation of the APM that is always up to date.

Milestones

Develop the Central Configuration Inversion Library

- Implement the library in a language chosen by the development team. Seems like we are able to reuse the feature parity dashboard backend and implement everything there.
    - Done in Node.js
    - Feature parity dashboard done
    - Go in development
- Create a JSON-based structure that describes all configurations, including metadata (e.g., OS requirements, version, description). The draft is outlined in this document.
    - Done
- Implement a basic CRUD based backend that has a file based “database”. This is used to make sure it’s up and running fast. At a later point a proper database may be used.
    - Done (reusing the feature parity dashboard’s backend)
- The basic routes needed are (these should be later on defined more closely)
    - **POST /config/validation** (POST is used, due to the number of bytes the overall configuration might have, which can easily go beyond get parameter limits)
    - **POST /config**
    - **PUT /config**
    - **POST /feature**
    - **GET /config/:platform:**
        - Return the whole config set that is related to a specific platform
    - **GET /config**
        - Return the whole config set
    - **GET /config/:platform:**
        - Return the whole config set that is related to a specific platform
    - **GET /feature**
    - … multiple more… these should be defined by the team
- Accessibility: The backend initially uses pre-defined secrets to update the data from the CI and by developers. Reading is public. This includes the validation.
    - To add new entries or update them in the UI, users have to be logged into the feature parity dashboard
- The service has to be internal. So we also likely do not need access control / need minimal access control. All of this is available in the feature parity dashboard.
    - Done

Integrate CI Checks

- Implement a GitHub/gitlab Action in each tracer's repository to verify conformance.
    - Done in Node.js
- Create a Github/gitlab Action template that may be used in each tracer.
    - Done in https://gitlab.ddbuild.io/DataDog/apm-reliability/libdatadog-build
- Raise errors (not warnings) if a tracer uses an unrecognized configuration or deviates from the central library.
    - Done by activating the CI job in the individual tracer/SDK/platform

Automate Version Detection & Updates

- GitHub Actions validate tracer's local config list against the central library. This is unidirection (we only check that local configurations do exist in the central configuration inversion backend).
- Check source code for additional environment variables that are not included in the local configuration list / fail if one is accessed. In Node.js this is done with a minimal helper that just reads the supported-configurations.json file. We added a linter to prohibit access to environment variables. Instead, the helper is used to access these. The helper will filter all environment variables that start with DD_ (and OTEL_) but that are not listed in the supported-configurations.json file. That way the tests will fail and it is as if that environment variable was never declared. The developer therefore has to add it to the local file first. Improve the developer experience by adding debug logs or failing in a way that notifies the developer but that would not fail in production code.
    - All environment variables that do not start with DD_ or OTEL_ are passed through, since we only care about limiting our Datadog configurations and sometimes need to read others as well.
    - Environment variables used for testing/dogfooding or specific customers will be marked as "internal" in the config inversion backend.
    - CI job name `check_config_inversion_local_file`
- CI errors if discrepancies are detected.
- If a configuration was newly added, it is first placed in the central configuration inversion. The support range for the individual platform/language/tracer is not yet included. That is automatically added with another CI job.
    - The tags of the release branch are read and compare the supported-configurations.json files in between the latest and the former release.
    - If a new configuration was added in the local file, the CI job fires a REST call to the central configuration backend to add that supportedIn range for the individual platform.
    - The CI job `config_inversion_update_central` for this exists in https://gitlab.ddbuild.io/DataDog/apm-reliability/libdatadog-build
    - This supports single release lines and multiple release lines at the same time. It must be defined in the local CI job which the platform/tracer uses. The release branches have to be listed in the CI job.
    - The CI is triggered by typical version commit names (matching /`v?\d+\.d+\.d+$/).

    - This also works for removing support.
    - We ignore pre-releases

Considerations

- Simplicity, and strict validation.
- A configuration version specifies the behavior. If a tracer deviates from that behavior, it has to use another version.
- Older configurations are not supported. The documentation would profit but we just wait till all users migrate to a current version. The old documentation has to stay around until then. A user should switch between the different documentations by choosing the right tracer version.
- The Feature Parity Dashboard could potentially show this data at some point.
    - It lists the supported configurations now

Alternatives

- Maintain current independent configurations (not recommended. This is the current status quo).
- Manual reviews (error-prone and time consuming. This is the current status quo).

Questions

1. What do we define as library version?
    a. Semver
        i. Patch: we add library support to an already existing environment variable / configuration
        ii. Minor: we add a new environment variable / configuration.
        iii. Major: we change the central configs config inversion schema

Proposed JSON Structure

```javascript
{
  "libraryVersion": "1.0.3",
  "features": {
    "AWS Payload extraction": {
      "description": "Foobar",
```

```
      "configurations": [{ "DD_TRACE_AWS_PAYLOAD": ["versionA",
"versionB", "versionC"], "DD_TRACE_AWS_PAYLOAD_SIZE": [],
"DD_TRACE_AWS_PAYLOAD_FREQUENCY": []}],
      "parent": null,
      "useCases": ["This is a great use case to use this feature.
It is written in Markdown to later on be rendered somewhere in a
nice way."],
    }
  },
  "configurations": {
    "DD_TRACE_ENABLED": [
      {
        "version": "RandomUnique", // mandatory
        "description": "Enables or disables tracing.", //
mandatory
        "supportedIn": {
          "go": { "from": "1.2.3", "to": "latest", "broken": [{
"from": "1.5.9", "to": "1.6.0" }], "requirements": ["linux",
"windows", "mac", "i64", "node > 20.10.2"]}, // optional (from
and to are mandatory, broken and requirements are optional)
          "python": {"from": "0.30.0", "to": "latest",
"requirements": ["linux", "windows", "arm"]},
          ".net": { "from": "3.13.0", "to": "e" }
        }, // mandatory
        telemetry_names": ["trace.enabled", "trace_enabled",
"enabled.trace"], // optional?
        "telemetry_value": "trace_enabled", // optional?"
        "impact": ["CPU", "Memory usage", "Disk space", "Data
frequency", "Correctness"], // optional?
        "product": ["Tracer", "RUM"], // mandatory
        "requiredENV": ["DD_SERVICE_NAME"], // optional
        "default": true, // mandatory
        "deprecated": false // optional
        "alias": [],
        "static": true, // Either dynamic (may change during
runtime) or not
```

```
        "experimental": false,
        "activatesEnv": [], // Optional. Hopefully empty!!!
        "format": "validation.pattern.regexp? types (Boolean,
Integers [min / max], String, ...)? JSON-schema?" // optional in
the beginning. Later on it's mandatory
      }
    ],
    "DD_LOG_LEVEL": [
      {
        "version": "A",
        "description": "Defines log verbosity.",
        "supportedIn": {
          "nodejs": {"from": "4.10.1", "to": "latest",
"requirements": ["linux"]}
        },
        "impact": ["CPU", "Memory usage"],
        "product": ["Tracer"],
        "requiredENV": [],
        "default": "verbose"

      },
      {
        "version": "B",
        "description": "Defines log verbosity with different
format.",
        "supportedIn": {
          "go": {"from": "4.15.1", "to": "latest",
"requirements": ["linux", "windows"]}
        }
      }
    ]
  }
}
```

Explanation of the Chosen Structure

- **Object-based configurations** enable quick direct access and clear versioning history. This represents simple relational data.
- **"from"/"to" versioning** clarifies supported tracer version ranges. This may also handle multiple release lines as used in e.g., Node.js. This just has to be defined in the CI job.
- **OS requirements per tracer** handle language-specific constraints clearly. This should not yet be implemented.

Implementation Details

Local List of Configurations in Each Tracer

- Tracer maintains local JSON:

```
None
{
  "supportedConfigurations": {
    "DD_TRACE_ENABLED": ["configurationVersionA",
"configurationVersionB"], // multiple versions are supported by
checking the value? By checking for the presence of another
environment variable?
    "DD_LOG_LEVEL": ["versionB"],
    "BRYANS_NEW_ENV": ["versionA"]
  },
  "aliases": {
    "DD_TRACE": "DD_TRACE_ENABLED",
    "DD_ENABLED_TRACE": "DD_TRACE_ENABLED"
  }
}
```

- CI ensures these configurations exist in the central library.

Example Local Configuration (Go Tracer)

```
None
{
  "supportedConfigurations": {
    "DD_TRACE_ENABLED": ["versionA", "versionB"],
```

```
    "DD_LOG_LEVEL": ["versionC"],
    "DD_SERVICE_NAME": ["versionA"]
  }
}
```

Reliability

- Ensure central library availability for CI.
- Lightweight structure for reliable and fast processing.
- All code should be fully tested.

Runtime implementation

To achieve the full feature set, we have to find an implementation that works in the individual platform / tracer.
It is relatively easy to achieve the overall goal with the described helper method that is used to access environment variables instead of directly accessing environment variables.
This helper just returns `null` / `nil` / `undefined` in case the entry is not listed in the supported-configurations.json file or has an alias. Especially the alias handling that we also want to automate across platforms requires such helper anyway. Otherwise the alias handling would have to explicitly happen in code or use an alternative abstraction that does something similar.
During development, the helper may also throw an exception or similar to detect issues early on. That should not be part of the production code!
To safely ignore environment variables we don't list, we add an extra linter that does static code analysis over the repository and checks for all known environment variables. We also add dynamic pattern checks in the linter of known ones. That should mitigate any customer running into issues where an environment variable would not have a functionality anymore. This safeguard is in addition to our local tests and system tests which both should cover all environment variables anyway.
The runtime implementation should later on also be used to simplify our general configuration implementation such as handling default values and more.

Out of scope

- Programmatic configuration during e.g., library start (as done in Node.js). Later on, we make sure to have an environment variable for each programmatic configuration next to it, where possible (methods would be difficult).

- “OS Requirements” have to later on be defined and implemented. This RFC concentrates on the basics to already provide a good benefit as is.
- Website/API endpoint to configure these to have live updates instead of using PRs to the central configuration. The website would also use a database and the repository would not be used anymore. Instead, regular CRUD REST endpoints would be used.
- The format specification for configuration values happens at a later point.

Steps to add a new Feature / Configuration

1. We open a PR at the central configuration repository and add that configuration including its description etc. to the “database” (JSON file). It does not yet contain the version of any supported library.
2. You open the PR to implement the new feature in the library. Now, the CI detects the new configuration (environment variable) and checks if it knows about it. If yes, the CI is green. Otherwise, red.
3. When the PR is released, the CI updates the central configuration (if the website already exists, it would just call the endpoint) and adds the tracer to the supported ones. This is done by reading the tag and comparing the local configurations with the ones in the central inversion library.

Summary

This centralized configuration library ensures consistent and validated configurations across all tracer implementations, integrates seamlessly with existing testing systems, and provides a clear, maintainable approach to configuration management.